It doesn’t remember. It’s handed everything again.

What a session actually is, and why what has to last goes in a file

Florian Gärtner

Where we are today

	What this course is for
	Getting it running
	Try it out
	The terminal, and how to read what Claude does
	Lunch
	A working day with Claude Code
	What the model reads, predicts and weighs
	A model, a menu, and a loop
→	**It doesn’t remember. It’s handed everything again.** ← you are here
	Wrap, done at 17:00

Learning objectives

After this block you will be able to:

1. **Trace** any element of Claude's behaviour back to what put it in the input.
2. **Predict** whether an action costs a full re-read, and say which condition it broke.
3. **Decide** what belongs in an always-loaded file and what should be loaded on demand.

From every turn to a file

Every turn
the whole conversation
handed over again

→

The cache
makes that cheaper
not a memory

→

The context window
Claude Code summarises
something comes out

→

A file
outside the conversation
it outlasts the session

Anything that has to last goes in a file.

It doesn’t remember.
It’s handed everything again.

Something you did not type

Exercise

You are about to have Claude write one line in a file.

Then you will start a new session, ask Claude two questions whose answers you never gave in that session, and Claude will answer both correctly.

One line

Exercise

Start Claude in your project folder and ask:

> Create a file called `CLAUDE.md` in this folder, with one line: The secret word is pangolin.

Open the folder and look at the file. That is where it lives, and that is all it is.

Then type `/exit`, and start Claude again in the same folder.

You ask. Claude writes. You read it and decide.

Now ask Claude

Exercise

> What is the secret word?

> What is my field?

Claude answers correctly

In this session, you never typed either answer. You never told Claude to open a file. You never mentioned a file at all.

When did Claude read them?

What the model actually received

```
<|system|>
You are Claude Code, a coding tool
that runs in a terminal.
<|end|>

<|tools|>
read_file, edit_file, run_command
<|end|>

<|files|>
~/.claude/CLAUDE.md:
  Explain each technical term ...
~/.claude/about-me.md:
  Position and field: ...
./CLAUDE.md:
  The secret word is pangolin.
<|end|>

<|status|>
Context: 35% used.
<|end|>

<|user|>
What is the secret word?
<|end|>
```

Block	Who wrote it	When
`system`	Anthropic	fixed
`tools`	Anthropic, and your MCP servers	at session start
`files`	Claude drafts, you approve	read once, at session start
`status`	a program	written fresh for each message
`user`	you	just now

One string. Not a conversation.

There is no conversation in there. No memory, no history object, no session.

There is a single piece of text with markers in it, and the markers are text too.

> **You typed one of those five blocks**
>
> You had Claude write a second one earlier, in a file, and probably were not thinking about it at the time. The other three arrived without you. All five are input, and the model cannot tell which part you meant.

One word, two sizes

The context is all of it. The whole string. That is what “context window” and “context length” measure, and it is how this course uses the word.

Some documentation also says **project context** for the files block on its own, always with the qualifier.

> ⚠ **Watch for the unqualified version**
>
> If you read “the context” somewhere, it almost always means everything.

Where that shape comes from

> ⚠ **The markers are made up. The blocks are not.**
>
> `<|system|>` and the others are invented. Anthropic does not publish its format.

What is not made up, and holds everywhere:

1. It is **one flat string**, not a structure the model holds open.
2. The roles are **markers inside that string**.
3. It is assembled fresh and sent **whole**, every single turn.

See it yourself, in any session. `/context` shows what fills the session, and `/context all` names the files. With your kit, they are `CLAUDE.md`, `about-me.md` and `rules/rules.md`. Claude can read you the system prompt and your files from the session file.

When Claude reads a file

```
<|assistant|>
I'll look at the cleaning script.
read_file("01_clean.R")
<|end|>

<|tool_result|>
library(dplyr)
raw <- read.csv("data/raw.csv")
...
<|end|>
```

The tool result is the whole file: **all 300 lines, word for word.**

> ⚠ **And it stays there**
>
> Every later turn re-sends those 300 lines, because they are now part of the string.

Turn two

The string from before, one line per block, one turn later:

```
┌─ the input, turn 2 ──────────────────────────────┐
│ SYSTEM     instructions, list of tools           │
│ FILES      ~/.claude/CLAUDE.md · ./CLAUDE.md     │
│ STATUS     written for turn one                  │
│ YOU        What is the secret word?              │
│ CLAUDE     The secret word is pangolin.          │
│ STATUS     written for turn two                  │
│ YOU        What is my field?                     │
└──────────────────────────────────────────────────┘
```

Nothing learned

Between turn one and turn two, the model learned nothing.

The weights do not change. All the model has of your conversation is its text, and every turn the whole text goes in again.

> **It feels like memory because the whole conversation is in front of it**
>
> Not because anything was remembered.

A notebook, not a memory

Most of us picture memory as something kept inside, plus recall that happens on its own.

The model has neither half. Claude Code writes your conversation down and hands it to the model again every turn.

> **That is a notebook**
>
> Taking notes puts part of your memory on paper, and you remember by reading.

Enter the cache

The same text, every turn

Claude is handed the whole conversation on every turn. If Claude processed it all from scratch each time, you would pay for all of it again on every turn.

By turn thirty, your first message would have been processed thirty times.

Keep the intermediate results

As the model reads, it works out intermediate results for every token, and then throws them away.

Caching is the decision not to throw them away. The next turn starts from those results and computes only what is new.

> ⚠ **The cache is not a memory**
>
> It is a discount on re-reading. Nothing is remembered. The same text goes in every time. Some of the work of reading it is skipped.

Why the results can be kept

The model works out its results token by token. The results for each token depend only on that token and the tokens before it, never on what comes after.

So when a turn adds text at the end, the results for everything before it are exactly what they were. Working them out again would give the same.

When you get the discount

Two conditions:

1. A stored entry matches the **start** of this input **byte for byte**, under **this model**.
2. That entry has not idled out.

Two conditions, three symptoms

What you did	Which condition broke
Stepped away and came back	2, it expired
Switched model	1, nothing was stored for that model
Spawned a subagent	1, the subagent's input starts with its own instructions

Only the first row loses a stored entry. In the other two, nothing stored matches the new input.

Where the change happens matters

The match starts at the beginning

Change something early and everything after it has to be processed again. Change something at the end and everything before it is still cached.

Claude Code arranges this for you by adding everything new at the end: your next message, every file Claude reads, every tool result. So an ordinary turn keeps its discount.

How long the cache lasts

> ⚠ **These numbers date. The two conditions do not.**
>
> Verified 2026-09-10. Check before you rely on them.

	On a subscription, within plan	Paying per token
Your main conversation	one hour	five minutes
Subagents, forks, compaction	five minutes	five minutes

The clock is an idle clock. Every turn resets it, free.

See it yourself, in any session. `/usage` shows how much of the session's input came from the cache, the misses, and whether the cache is warm right now.

Cached is not shorter

The cache skips work, but every word of the conversation is still in front of the model. Text that does not matter still competes with the text that does.

A long session works worse, on average, whether it is cached or not.

Caching makes a long conversation cheaper. It does not make it better.

When the notebook is full

The notebook has a page limit

Every model has a fixed limit on how much input it can take: its **context window**.

The conversation only grows. Every turn adds to it and nothing is ever removed.

So what happens when it fills up?

Claude Code summarises it for you

When the conversation is nearly full, Claude Code compacts it on its own. Claude writes a summary of the older history, the summary replaces it, and the session carries on.

The summary keeps what Claude judges worth keeping, which may not be what you would keep.

> ⚠ **Something has to come out**
>
> If you do not choose what, Claude Code chooses for you.

Two ways under the limit

`/clear` throws the conversation away and starts over.

`/compact` replaces it with a summary of itself and carries on. You can tell `/compact` what to keep: `/compact keep the variable definitions`.

> ⚠ **What each one costs**
>
> `/clear` costs nothing but the conversation. `/compact` costs one request that writes the summary, read mostly from the cache, and you keep only what the summary keeps.

So it goes in a file

Nothing in the window survives

Sooner or later the conversation goes. You clear it, you compact it, or Claude Code compacts it for you: near the limit, and on some accounts during a long break.

Anything that has to outlive this session cannot live in the session.

In a file, it lasts

The conversation and a `CLAUDE.md` file reach Claude the same way: as text in its input.

	who writes it	**how long it lasts**
the conversation	you and Claude, automatically	until it is cleared or compacted
a file	Claude, when you ask (`/session-end`)	until you delete it

> **A file is not a workaround**
>
> It is the same notebook, written on purpose, and it stays.

Two ways a file gets in

Your instruction files are read when the session starts and handed to Claude with every turn: the `CLAUDE.md` files (one in a subfolder waits until Claude reads a file there), any file they name with `@`, as your kit's `CLAUDE.md` names `about-me.md`, and anything in a `.claude/rules/` folder.

Everything else gets in only when Claude reads it: your notes, a paper converted to markdown, a codebook, the notes Claude keeps for itself.

- **Always loaded** is always there, and takes up room in every turn, whether you needed it or not.
- **On demand** takes no room until Claude reads it, which happens only if something leads Claude there. From then on it stays in the conversation, like every file Claude reads.

Which of your own project notes belong in `CLAUDE.md`?

What to take away

1. It does not remember. It is handed the whole conversation again, every turn.
2. The cache makes that cheaper. It does not make it a memory.
3. The conversation has a hard limit, and near it Claude Code compacts it into a summary.
4. So anything that has to last goes in a file.
5. Always loaded is always there, and takes up room in every turn. On demand takes none, and is read only if something leads Claude there.

Before you go

Your project · free time

- Type `/session-end` in your project's session.
- Open the file Claude wrote, `claude/next_steps.md`. Could a new session tell from it, tomorrow, where you stopped?